Darshan UNIVERSITY

योग: कर्मसु कौशलम्

Python Programming - 24010101116

Lab - 12

Roll No : 366

Name : Kachhava Nakul U.

Exception Handling

01) WAP to handle following exceptions:

1. ZeroDivisionError
2. ValueError
3. TypeError #### Note: handle them using separate except blocks and also using single except block too.

In [16]:
```
try:
    n = int(input("Enter a number n: "))
    m = int(input("Enter a number m: "))
    ans = n/m
    s = "Hello" + 10
except ZeroDivisionError as err:
    print(type(err).__name__,":",err)
except ValueError as err:
    print(type(err).__name__,":",err)
except TypeError as err:
    print(type(err).__name__,":",err)
except Exception as err:
    print(type(err).__name__,":",err)
```

```
Enter a number n: 10
Enter a number m: 0
ZeroDivisionError : division by zero
```

02) WAP to handle following exceptions:

1. IndexError
2. KeyError

In [13]:
```
try:
    a = [1,2,3]
    i = int(input("Enter index:"))
    a[i]
    d = {1: "abc", 2: "bcd"}
    k = int(input("Enter key:"))
    d[k]
except IndexError as err:
    print(type(err).__name__,":",err)
except KeyError as err:
    print(type(err).__name__,":",err)
```

```
Enter index:2
Enter key:5
KeyError : 5
```

03) WAP to handle following exceptions:

1. FileNotFoundError
2. ModuleNotFoundError

In [21]:
```
try:
    import xyz
    fp = open("xyz.txt","r")
    fp.close()
except FileNotFoundError as err:
    print(type(err).__name__,":",err)
except ModuleNotFoundError as err:
    print(type(err).__name__,":",err)
```

```
FileNotFoundError : [Errno 2] No such file or directory: 'xyz.txt'
```

04) WAP that handles all type of exceptions in a single except block in standard error message format.

In [23]:
```
try:
    a = [1,2,3]
    i = int(input("Enter index:"))
    a[i]
    10/0
except Exception as err:
    print(type(err).__name__,":",err)
```

```
Enter index:5
IndexError : list index out of range
```

05) WAP to demonstrate else and finally block.

In [26]:
```
try:
    a = int(input("Enter a:"))
    b = int(input("Enter b:"))
    c = a/b
except Exception as err:
```

```
    print(type(err).__name__,":",err)
else:
    print("Ans: ",c)
finally:
    print("Code Execution Completed.")
```

```
Enter a:5
Enter b:5
Ans:  1.0
Code Execution Completed.
```

06) WAP to create an udf divide(a,b) that handles ZeroDivisionError.

In [30]:
```
def divide(a,b):
    try:
        c = a/b
    except ZeroDivisionError as err:
        print(type(err).__name__,":",err)
    else:
        return c

a = int(input("Enter a:"))
b = int(input("Enter b:"))
divide(a,b)
```

```
Enter a:10
Enter b:0
ZeroDivisionError : division by zero
```

07) WAP to accept item prices and calculate total. Raise ValueError if price is negative, otherwise print the total bill amount.

In [32]:
```
try:
    sum1 = 0
    n = int(input("Enter number of items: "))
    for i in range(n):
        p = int(input(f"Enter item {i+1} price: "))
        if p < 0:
            raise ValueError("Price can not be negative.")
        sum1 += p
except ValueError as err:
    print(type(err).__name__,":",err)
else:
    print(f"Total Bill: {sum1}")
```

```
Enter number of items: 2
Enter item 1 price: 12
Enter item 2 price: -15
ValueError : Price can not be negative.
```

08) Create a short program that prompts the user for a list of grades separated by commas.

Split the string into individual grades and use a list comprehension to convert each string to an integer.

You should use a try statement to inform the user when the values they entered cannot be converted.

In [44]:
```
try:
    g = input("Enter Grades seperated by commas: ")
    li = [int(i) for i in g.split(",")]
except Exception as err:
    print("String Values can not be converted into integer.")
else:
    print(li)
```

```
Enter Grades seperated by commas: 1,abc,2,3
String Values can not be converted into integer.
```

09) Accept 5 subject marks (0-100). Use assert for validation.

Sample Input (Error Case)

Enter marks: 110

Sample Output (Error Case)

AssertionError: Marks should be between 0 and 100.

Sample Input (Normal Case)

Valid marks entered

Sample Output (Normal Case)

Average Marks displayed

In [49]:
```
try:
    sum2 = 0
    for i in range(5):
        m = int(input(f"Enter Subject {i+1} marks: "))
        assert 0 <= m <= 100
        sum2 += m
except AssertionError as err:
    print(type(err).__name__,":","Marks should be between 0 and 100.")
else:
    print("Valid marks entered.")
    print(f"Average Marks: {sum2//5}")
```

```
Enter Subject 1 marks: 56
Enter Subject 2 marks: 65
Enter Subject 3 marks: 67
Enter Subject 4 marks: 100
Enter Subject 5 marks: 111
AssertionError : Marks should be between 0 and 100.
```

10) WAP that gets password from user. Password must have at least 8 characters and one digit.

Raise WeakPasswordError if invalid, print the messsage "Correct" otherwise.

In [55]:
```
class WeakPasswordError(Exception):
    pass

try:
    s = input("Enter password: ")
    c = 0
```

```
    for i in s:
        if i.isdigit():
            c += 1
    if not(len(s) >= 8 and c > 0):
        raise WeakPasswordError("Password must have at least 8 characters and one d
except WeakPasswordError as err:
    print(type(err).__name__,":",err)
else:
    print("Correct.")
```

```
Enter password: abc
WeakPasswordError : Password must have at least 8 characters and one digit.
```

11) WAP to raise your custom Exception named NegativeNumberError with the error message : "Cannot calculate the square root of a negative number" :

if the given number is negative.

otherwise print the square root of the given number.

In [60]:

```
from math import sqrt

class NegativeNumberError(Exception):
    pass

try:
    sq = int(input("Enter Square of a number: "))
    if sq < 0:
        raise NegativeNumberError("Cannot calculate the square root of a negative r
except NegativeNumberError as err:
    print(type(err).__name__,":",err)
else:
    print(f"Square root: {sqrt(sq)}")
```

```
Enter Square of a number: -2
NegativeNumberError : Cannot calculate the square root of a negative number.
```

In [ ]: